PROJECT TUTORIAL

Junaid Ahmad Noor [24158148]
GitHub: https://github.com/JunaidAhmadNoor/ML-Tutorial

How Self-Attention Solves the Long-Range Dependency Problem

A Tutorial Comparing RNNs and Transformer Self-Attention

1. What is the Long-Range Dependency Problem

In the world of Machine Learning, we mostly deal with data that lives in sequences. Think of a sentence, a paragraph or a whole document. To understand the whole document, you must make some connections that connect the parts of chunks. Before going to any code part or Formulas, let me explain the actual problem we are trying to solve.

For Example:
"The book did not fit in the bag because it was too heavy."

In this what "It" refers to? **The book** Right? You get it instantly. Now think about how your brain processes it. It goes back and finds the word that makes sense logically and chooses it. Now think about a Document where you brain also struggles. For machine learning models, those who read text a word at a time, this is difficult for them. There are different words between "it" and "book". So, the gap between where the information appears and where it is exactly needed, is called Long-Range dependency in technical terms.

The Longer the distance the model struggles, for a sentence it manages, but for a paragraph, for a document, things start to fall apart. So, this tutorial will take you through the era of Recurrent Neural Networks (RNNs), the reason it "failed" at longer distances, and how the Self-Attention mechanism was the mathematical bridge we needed.

2. How RNNs Process Sequences

Before we move towards the solution, let us discuss the Recurrent Neural Networks (RNNs). RNNs idea is beautiful in its simplicity, it processes a sequence one step at a time: read one token at a time and pass a "memory" forward from each step to the next. This memory is called the hidden state, written as h_t.

The update rule looks like this:

$$h_t = \tanh(W_h * h_{t-1} + W_x * x_t)$$

It takes an input (x_t), combines it with a "Hidden State" (h_{t-1}) from the previous step, and produces a new Hidden State (h_t).

The problem? Well, everything that has ever happened in the past has to fit into this one vector. You could think of it as trying to describe the entire book in one sentence. It's just not possible. Furthermore, the further back in the past something is, the further it is diluted by everything else that has ever happened in the meantime.

There are two big issues here:

- Vanishing Gradients: As the gradient backpropagation occurs over a long time, the gradients eventually approach zero exponentially. By the time the gradients make it back to the beginning of the sequence, they are zero. Therefore, the model cannot learn the importance of the beginning elements in the sequence. **(Pascanu et al., 2013)**

- Information Bottleneck: A vector has a certain size, no matter how long the sequence is. A 5-element sequence and a 500-element sequence are equivalent in vector size. Something must be lost.

Long Short-Term Memory networks (LSTMs) were designed to address the problem. They provide a gating mechanism, which consists of the input gate, forget gate, and output gate. This allows the model to have more control over the information it processes and the information it ignores. This has a great effect on medium-length sequences.

3. The Copy Task: Our Experiment

To make this difference or comparison measurable, we use Copy Task. It was introduced by Graves et al. in 2014 as a benchmark for testing memory in neural networks.
Its work is simple, model receives a sequence of random tokens, followed by some padding, followed by a GO signal. Its job is to reproduce the original tokens in the output positions. Like this:

Input: [8, 7, 5, 4, PAD, PAD, PAD, PAD, GO]
Target: [0, 0, 0, 0, 0, 8, 7, 5, 4]

The reason this works well as a test is that the model cannot guess based on meaning or grammar, it simply must remember what it saw first then reproduce it later. This means model has no shortcuts. If a model struggles with this means it is struggling with memory and trying to maintain information over a gap. We test both models on different sequences lengths: 5, 10, 20, 30, 40 and 50 tokens. Short sequences are easy for everyone as compared to large sequences where the whole differences emerge.

4. Self-Attention the Core Idea

Self-Attention works totally different from RNNs. It looks at the whole sequence at once and figures out the direct relationship between every pair of positions, but we know RNNs reading one token at a time and passing a hidden state forward.

The mathematical formulation was introduced by **Vaswani et al. in the landmark 2017 paper "Attention Is All You Need":**

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$$

Imagine you are in a public place and looking for someone to talk about a specific topic. You have something in mind you want to discuss; that's your Query(Q). Around you, people are having conversations and you are listening to some keywords they are mentioned; those are keys(K). When you hear someone talking about something relevant in your mind, you focus on their discussion and want to listen to what they are saying, that's the Value(V)
Self-attention does exactly this for every token simultaneously. Each token generates three vectors:

- **Query (Q)** = what you're looking for?
- **Keys (K)** = clues that help you decide who is relevant?
- **Values (V)** = the actual information you take in?

The dot product QK^T indicates how relevant each word is to all the other words. SoftMax normalizes this so that it adds up to one. Then, the Value vectors are weighted and added together, giving each word a new representation that includes context from all the sequence.

The division by sqrt(d_k) is used to prevent the values from growing too large. Without this division, the SoftMax function would result in a highly peak distribution. This causes the gradients to vanish.

4.1. Multiple Head Attention

The concept of multi-head attention is that the model can examine a sentence in a variety of different ways at once, rather than just one. Each of these different "heads" of attention is looking at a different aspect of the input. For example, one head looking at grammar, another one looking at how words are related with each other with large gaps, and another one looking at how words are related to each other if they are close. When all the heads of attention looked at input, their findings are combined.

4.2. Why Positional Encoding is Necessary

Self-Attention does not understand the order of words. It treats the words as they have no fixed position which means if you change the positions of the words in a sentence it does not get anything changed. For example, "Book in a shelf" and "shelf in a book" have totally different meanings but they are using same words. To solve this issue, we add positional embeddings to each word before applying the attention.

out = token_embedding(x) + positional_embedding(position)

The positional embedding gives every word information about its position in the sentence, so, the model knows which word comes first, second and so on. In our code, these positional embeddings are learned during training which means model figures out the best way to represent positions on their own.

5. Path Length: The Theoretical Advantage

One of the best ways to understand why self-attention handles the long-range dependencies better as compared to RNNs is through the concept of path length, how many steps information must travel between two positions.

In RNNs, if you want some information between position 1 to position n, then signal pass through n-1 hidden states. The path length is **O(n).** Each intervening state is a point at which information could be lost.

In self-attention, every token can directly attend to every other token in just single operation. The path length between two positions in the input is **O(1),** no matter how distant they are in the sequence. The connection is direct.

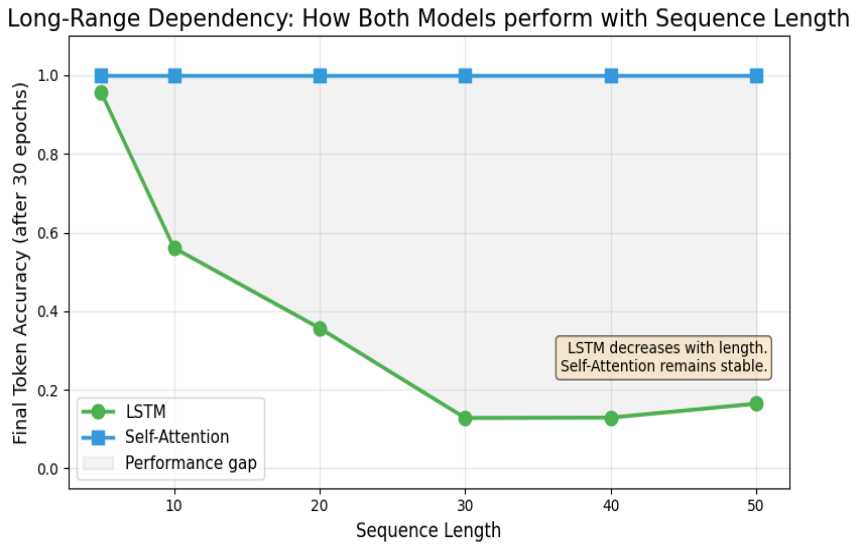
6. Experimental Results

We trained both the LSTM and self-attention model on the copy task in all six sequences. Both models used the same embedding dimensions and same number of training samples (2000) and same number of epochs to get fair comparison.

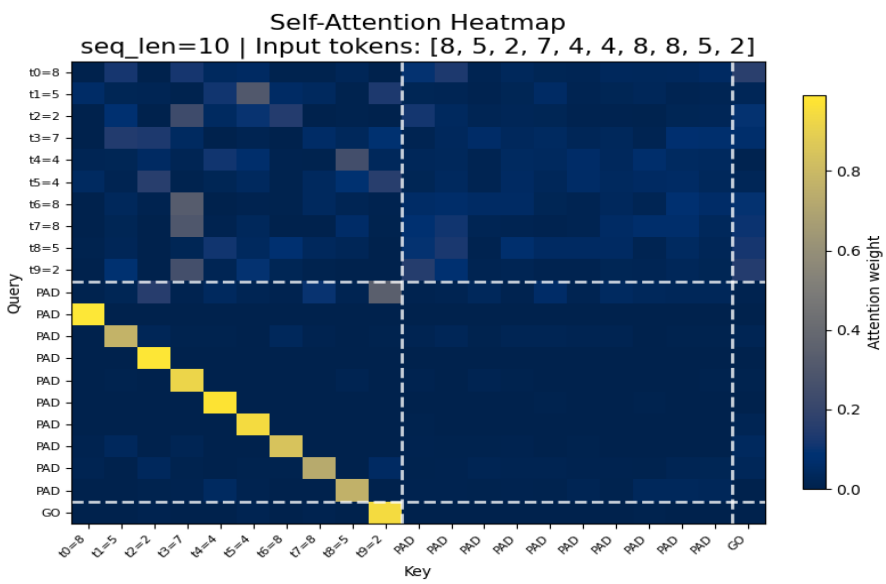
We found these results after training:

Sequence Length	LSTM Accuracy	Self-Attention Accuracy	Winner
5	0.956	1.000	Self-Attention
10	0.561	1.000	Self-Attention
20	0.357	1.000	Self-Attention
30	0.128	1.000	Self-Attention
40	0.129	1.000	Self-Attention
50	0.165	1.000	Self-Attention

Results show a clear difference between LSTM and self-attention. Even in short sequences, LSTM is not perfect, on the other hand self-attention achieves 100% results. As move forward, sequences get longer, LSTMs performance drops a lot and on 30 it performs very poorly, like it is doing random guessing. On the other hand, self-attention performs 100% in all the sequences.



We also visualized how the models work internally. For instance, in the self-attention model, it is very clear that there is a diagonal pattern in the attention weights. This shows that each output can focus on the corresponding input. However, this is not same for LSTM, where it is seen that the hidden state is changing over time, and information from before is overwritten by new information.



7. Limitations of Self-Attention

Self-attention is good, but it is not perfect, it has some limitations also, and these help you to make better decision.

- Quadratic complexity: Attention matrix is T * T, so memory grows as O(T^2). Like for 1000 sequence tokens, 1,000,000 attention scores per head per layer were created. For very long sequences it becomes impractical on standard hardware.
- Large model size: Transformers require significant hardware resources because multi-head attention with multiple layers introduces a lot of parameters.
- Struggles with extremely long sequences: Very long sequences still need specialized architecture.

These limitations led to new and more efficient versions of attention. For example, Longformer reduces heavy computation by only focusing on certain parts of the sequence instead of everything. Reformers use a clever method to pay attention in a faster and cheaper way.

The main point is that self-attention is not always the best. It works well when you need to understand long connections in data and have enough data and computing power.

8. Summary

RNNs read sequences one step at a time and struggle with long sequences. LSTMs help but don't fully fix it. Self-attention connects all tokens directly, keeps full accuracy even on long sequences, and uses queries, keys, values, multi-head attention, and positional encoding. It's powerful but needs more memory, data, and computers.
Our experiments verify this claim, where self-attention scored 100% accuracy for all sequence lengths, whereas LSTM scored as low as 13% at length 30.

Accessibility

Colorblind-friendly color palettes are applied in all figures. The "cividis" colormap is selected for the attention heatmap for uniformity of perception, and "orange" and "red" are selected for the prediction grid to avoid any issues with red-green color blindness.

References

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. & Polosukhin, I., 2017. *Attention Is All You Need*. NeurIPS. Available at: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf [Accessed 15 February 2026].
2. Hochreiter, S. & Schmidhuber, J., 1997. *Long Short-Term Memory*. Neural Computation, 9(8), pp.1735–1780. Available at: https://www.bioinf.jku.at/publications/older/2604.pdf [Accessed 22 February 2026].
3. Pascanu, R., Mikolov, T. & Bengio, Y., 2013. *On the difficulty of training recurrent neural networks*. ICML. Available at: https://proceedings.mlr.press/v28/pascanu13.html [Accessed 28 February 2026].
4. Olah, C., 2015. *Understanding LSTM Networks*. Available at: https://colah.github.io/posts/2015-08-Understanding-LSTMs/ [Accessed 3 March 2026].
5. Alammar, J., 2018. *The Illustrated Transformer*. Available at: https://jalammar.github.io/illustrated-transformer/ [Accessed 5 March 2026].
6. Graves, A., Wayne, G. & Danihelka, I., 2014. *Neural Turing Machines*. arXiv:1410.5401. Available at: https://arxiv.org/abs/1410.5401 [Accessed 15 March 2026].